

IF2224 FORMAL LANGUAGE AND AUTOMATA THEORY

Arion Compiler - Milestone 3: Semantic Analysis

May 20, 2026



Made Branenda Jordhy - 13524026

Muhammad Nur Majiid - 13524028

Jason Edward Salim - 13524034

Athilla Zaidan Zidna Fann - 13524068

School of Electrical Engineering and Informatics

Institut Teknologi Bandung

Semester II 2025/2026

Contents

1	Theoretical Background	3
1.1	Semantic Analysis	3
1.2	Abstract Syntax Tree (AST)	3
1.3	Decorated AST	4
1.4	Attributed Grammar dan Syntax-Directed Translation	4
1.5	Symbol Table: <code>tab</code> , <code>btabs</code> , dan <code>atab</code>	5
1.6	Scope Resolution dan Display	6
1.7	Visitor Pattern dan Traversal DFS	6
1.8	Type Compatibility	6
2	Design and Implementation	7
2.1	Overview Pipeline	7
2.2	AST Node Hierarchy	7
2.3	Struktur Symbol Table	8
2.4	Inisialisasi: Reserved Slot dan Predefined Identifier	9
2.5	Type Compatibility Rules	9
2.6	Semantic Visitor Architecture	10
2.7	Scope Management	10
2.8	Alur Visit Function untuk Konstruksi Kunci	10
2.9	Inferensi Tipe pada Operator Biner	11
2.10	Penanganan Prosedur, Fungsi, dan Parameter	12
2.11	Error Handling	12
2.12	Format Decorated AST	13
3	Testing	14
3.1	Ringkasan Kasus Uji Valid	14
3.2	Ringkasan Kasus Uji Error	15
3.3	Hasil Pengujian Valid	15
3.4	Hasil Pengujian Error	33
4	Conclusion	38
4.1	Saran	38
	Appendix	41

1 Theoretical Background

1.1 Semantic Analysis

Semantic analysis adalah fase ketiga dalam pipeline kompilasi, setelah *lexical analysis* (Milestone 1) dan *syntax analysis* (Milestone 2). Pada tahap ini, parse tree yang sudah dibangun parser ditelusuri ulang untuk memverifikasi **makna** program, bukan lagi sekadar bentuk sintaktiknya. Pengecekan yang dilakukan meliputi empat aspek besar:

- **Type checking** - memastikan operator dan operand memiliki tipe yang kompatibel (mis. tidak menjumlahkan `integer` dengan `string`).
- **Symbol table management** - memastikan setiap identifier dideklarasikan sebelum digunakan dan tidak dideklarasikan ulang dalam cakupan yang sama.
- **Scope resolution** - menentukan validitas akses variabel berdasarkan hierarki blok kode (global vs lokal vs nested).
- **Control flow validation** - memastikan struktur alur kontrol logis (kondisi `if/while` bertipe `boolean`, dsb.).

Sebagai keluaran, semantic analyzer menghasilkan **Decorated Abstract Syntax Tree (AST)** - yaitu AST yang setiap nodenya dianotasi dengan informasi tambahan seperti tipe data hasil ekspresi, indeks ke symbol table, dan *lexical level* tempat node tersebut berada.

1.2 Abstract Syntax Tree (AST)

Abstract Syntax Tree adalah bentuk ringkas dari parse tree yang hanya menyimpan informasi struktural yang relevan secara semantis. Parse tree masih memuat seluruh non-terminal antara (`<simple-expression>`, `<term>`, dst.) serta token sintaktis (`begin`, `end`, `;`, `(`, `)`) yang berguna untuk pengecekan grammar tetapi tidak diperlukan untuk analisis makna. AST membuang informasi tersebut dan mempertahankan hanya hubungan antar operator-operand.

Figure 1 memperlihatkan perbedaan tersebut untuk ekspresi sederhana $7 + (2 + 3)$: parse tree memuat tujuh tingkat non-terminal sebelum mencapai daun, sedangkan AST cukup direpresentasikan oleh tiga node.

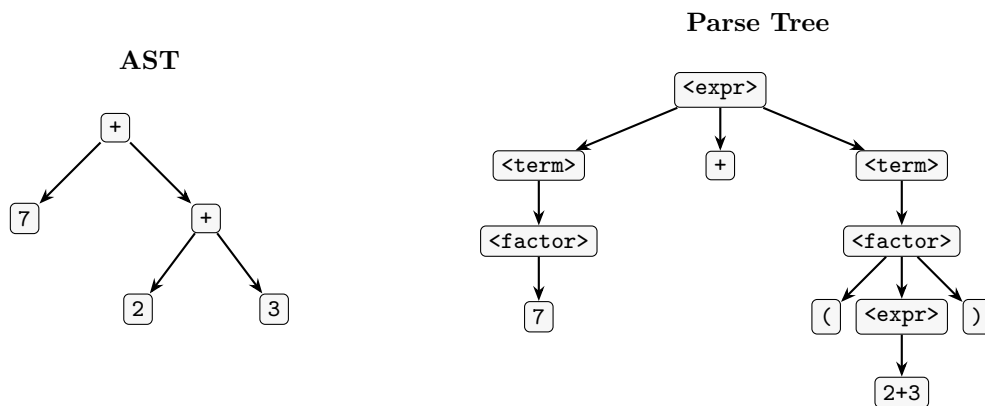


Figure 1: Perbandingan AST dan parse tree untuk ekspresi $7 + (2 + 3)$. AST membuang non-terminal antara dan token sintaktis, menyisakan hanya operator-operand yang relevan secara semantis.

1.3 Decorated AST

Decorated AST adalah AST hasil tahap semantik di mana setiap node telah dilengkapi dengan *atribut* hasil evaluasi. Atribut minimal yang ditanamkan pada implementasi Arion adalah:

- **dataType** - tipe data akhir dari ekspresi node tersebut (mis. `integer`, `real`, `boolean`).
- **tabIndex** - indeks ke entri `tab` pada symbol table (untuk identifier, ini adalah deklarasi yang di-*resolve*).
- **lexLevel** - tingkat *lexical scope* tempat node berada (0 = global, 1 = dalam prosedur/fungsi tingkat-1, dst.).

Anotasi inilah yang akan menjadi masukan utama untuk tahap berikutnya (*intermediate representation generation* dan *code generation*).

1.4 Attributed Grammar dan Syntax-Directed Translation

Attributed grammar adalah CFG yang dilengkapi dengan *atribut* pada setiap simbol dan *semantic rule* yang menentukan bagaimana atribut tersebut dihitung. Untuk pengerjaan Milestone 3 ini, atribut yang digunakan bersifat **synthesized** (dihitung dari atribut child ke parent) dengan beberapa atribut **inherited** (mis. `lexLevel`) yang diturunkan dari parent ke child. Karakteristik ini menjadikannya *L-Attributed Grammar*, yang cocok dievaluasi dengan satu kali penelusuran DFS dari kiri ke kanan.

Tabel 1 memperlihatkan contoh aturan semantik yang digunakan pada beberapa produksi penting.

Production Rule	Semantic Rule
<code><assignment-statement> → ID := <expr></code>	<code>node = new AssignNode(new VarNode(ID.lexeme), expr.node)</code>
<code><expression> → <simple-expr> + <term></code>	<code>node = new BinOpNode('+', simple_expr.node, term.node)</code>
<code><expression> → <simple-expr></code>	<code>node = simple_expr.node</code>
<code><factor> → INTCON</code>	<code>node = new NumberNode(INTCON.value, isReal=false)</code>
<code><factor> → ID</code>	<code>node = new VarNode(ID.lexeme)</code>
<code><term> → <factor></code>	<code>node = factor.node</code>
<code><factor> → STRING_LITERAL</code>	<code>node = new StringNode(STRING_LITERAL.value)</code>
<code><procedure-call> → ID (<params>)</code>	<code>node = new CallNode(ID.lexeme, params.nodes)</code>

Table 1: Contoh *semantic rule* yang digunakan untuk konstruksi AST.

1.5 Symbol Table: tab, btab, dan atab

Sesuai spesifikasi, symbol table pada Arion dipecah menjadi tiga tabel terpisah yang saling merujuk:

- **tab** - tabel identifier utama. Menyimpan konstanta, variabel, tipe, prosedur, fungsi, dan field record dengan atribut `id`, `link`, `obj`, `type`, `ref`, `nrm`, `lev`, `adr`.
- **btab** - *block table*. Setiap entri merepresentasikan satu blok (program global, prosedur, fungsi, atau definisi record). Atribut: `last`, `lpar`, `psze`, `vsze`.
- **atab** - *array table*. Setiap entri merepresentasikan tipe komposit array atau subrange. Atribut: `xtyp`, `etyp`, `eref`, `low`, `high`, `elsz`, `size`.

Figure 2 memperlihatkan hubungan antar ketiga tabel.

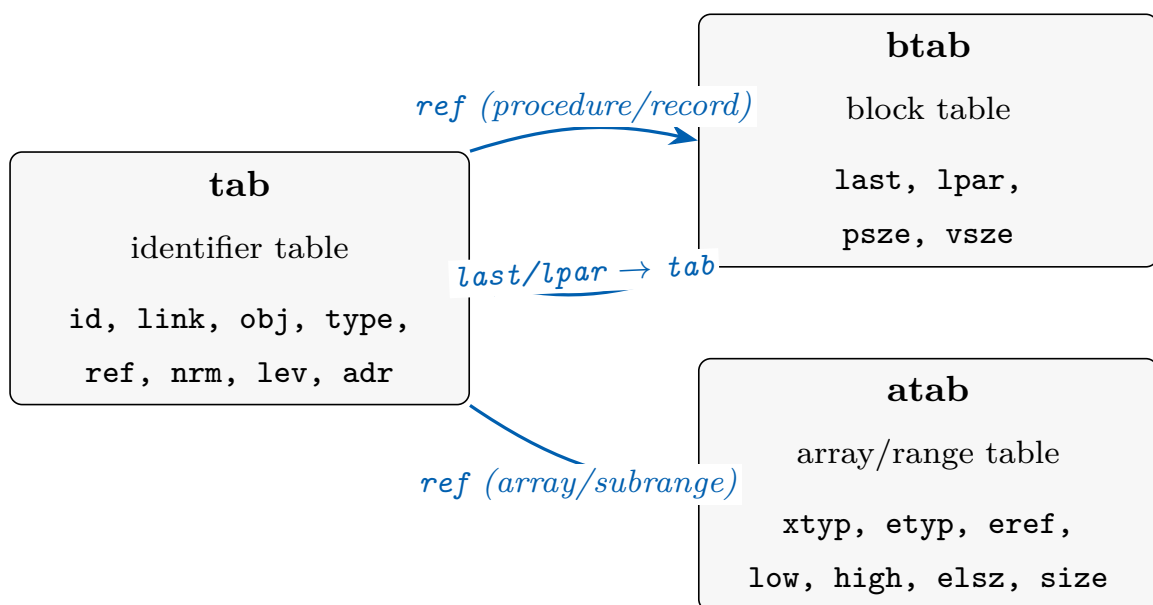


Figure 2: Hubungan antara **tab**, **btab**, dan **atab**. Field `ref` pada **tab** mengarah ke entri **btab** (untuk prosedur/fungsi/record) atau **atab** (untuk array/subrange).

Setiap blok membentuk **linked list** dari deklarasi yang dimilikinya: `btab[i].last` menunjuk ke entri terakhir di `tab` yang termasuk blok ke-`i`, dan masing-masing entri pada `tab` memiliki kolom `link` yang menunjuk ke deklarasi sebelumnya dalam blok yang sama. Penambahan deklarasi bersifat *push* ke kepala list - lihat Figure 3.



Figure 3: Tiap blok memuat *linked list* deklarasinya. Field `link` pada setiap entri `tab` menunjuk ke deklarasi sebelumnya dalam blok yang sama; entri terbaru selalu ditambahkan di kepala list lewat `btab[i].last`.

1.6 Scope Resolution dan Display

Scope dikelola dengan **stack display** yang menyimpan indeks `btab` dari setiap level cakupan yang sedang aktif. Pencarian identifier (*lookup*) dimulai dari level terdalam dan menelusuri ke level paling luar (level 0 = global, termasuk *reserved words* dan *predefined identifier*).

Figure 4 memperlihatkan keadaan `stack display` ketika analyzer berada di dalam prosedur `Inner` yang bersarang di dalam prosedur `Outer`.



Figure 4: `Stack display` ketika analyzer berada dalam `Inner`. Lookup identifier menelusuri stack dari level tertinggi (terdalam) ke level 0 (global).

1.7 Visitor Pattern dan Traversal DFS

Setiap jenis node AST memiliki fungsi `visit_<node>` yang dipanggil secara rekursif. Fungsi `visit` ini bertugas:

1. Memanggil `visit` pada child node-nya (penelusuran DFS).
2. Mengakses atau memperbarui symbol table (push/pop block, enter identifier, lookup).
3. Menghitung atribut atau tipe semantik node.
4. Menanamkan anotasi pada node (`dataType`, `tabIndex`, `lexLevel`).

1.8 Type Compatibility

Type compatibility rule menentukan kapan dua tipe dapat digunakan bersama dalam ekspresi atau *assignment*. Dua jenis kompatibilitas yang dibedakan:

- **Compatible** - dua tipe dapat dibandingkan atau dioperasikan (mis. pada operator relational).

- **Assignment-compatible** - tipe sumber dapat dikonversi ke tipe tujuan pada operasi `:=`, misalnya `integer` dapat di-assign ke variabel `real`.

Aturan dasar yang digunakan dirangkum pada Tabel 3 di Bagian Implementasi.

2 Design and Implementation

2.1 Overview Pipeline

Tahap semantik dijalankan sebagai kelanjutan langsung dari parser Milestone 2. Pipeline kompilasi sekarang terdiri atas lima komponen yang dirangkai pada `src/main.cpp`.

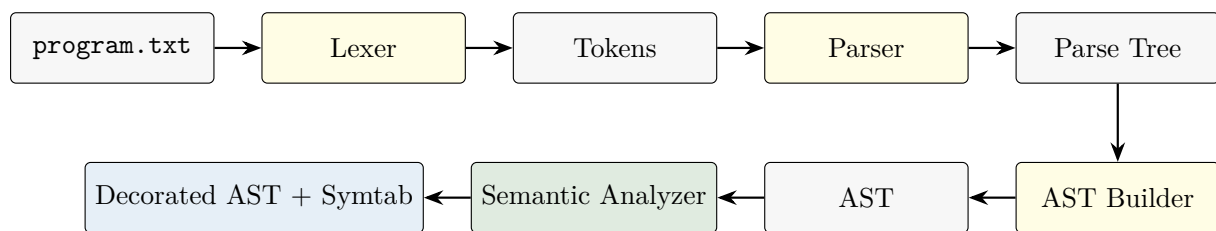


Figure 5: Pipeline kompilasi Arion sampai Milestone 3. Komponen baru yang ditambahkan pada milestone ini adalah *AST Builder* dan *Semantic Analyzer*, keduanya menghasilkan **Decorated AST** beserta isi symbol table sebagai keluaran tahap semantik.

Source code milestone ini terbagi menjadi modul berikut:

- `src/header/ast.hpp` & `src/lib/ast.cpp` - definisi dan implementasi node AST serta `buildAST()` yang mengkonversi parse tree menjadi AST.
- `src/header/syntab.hpp` & `src/lib/syntab.cpp` - definisi `SymbolTable` (`tab/btab/atab`) dan kelas `SemanticAnalyzer` dengan seluruh visit functions.
- `src/main.cpp` - menyambung lexer, parser, AST builder, dan semantic analyzer menjadi satu pipeline interaktif.

2.2 AST Node Hierarchy

AST direpresentasikan oleh basis struct `ASTNode` yang diperluas oleh berbagai subtype. Setiap subtype menyimpan informasi spesifik untuk konstruksi bahasa tertentu. Figure 6 memperlihatkan kategorisasi node.

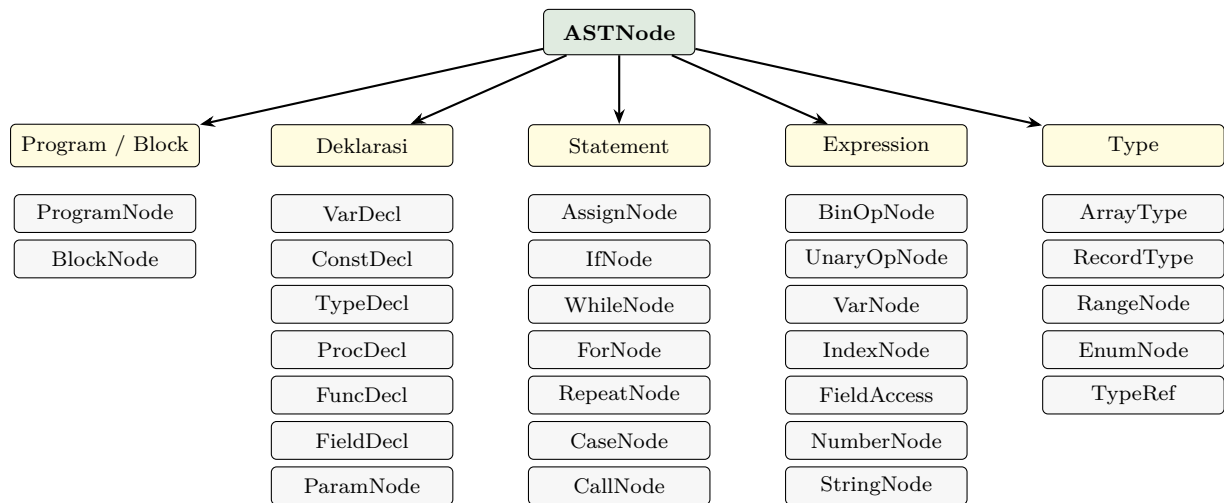


Figure 6: Hirarki node AST. Basis ASTNode memuat field anotasi (`dataType`, `tabIndex`, `lexLevel`, `line`) yang diisi oleh *semantic analyzer* sehingga menghasilkan *Decorated AST*. Lima kategori utama dikelompokkan pada kolom terpisah agar tidak terjadi tumpang tindih.

Setiap node mewarisi struktur dasar berikut:

```

1 struct ASTNode {
2     ASTNodeType nodeType; // jenis konstruksi (PROGRAM, ASSIGN, BINOP, ...)
3     DataType    dataType; // tipe hasil setelah analisis semantik
4     int         tabIndex; // indeks ke tab; -1 jika tidak relevan
5     int         lexLevel; // lexical level tempat node berada
6     int         line;     // baris sumber untuk pesan error
7 };
  
```

2.3 Struktur Symbol Table

Tiga tabel didefinisikan sebagai `std::vector` sehingga indeks setiap entri stabil dan dapat dijadikan referensi silang antar tabel.

```

1 struct TabEntry { std::string id; int link, obj, type, ref, nrm, lev, adr; };
2 struct BTabEntry { int last, lpar, psze, vsze; };
3 struct ATabEntry { int xtyp, etyp, eref, low, high, elsz, size; };
4
5 class SymbolTable {
6     std::vector<TabEntry> tab;
7     std::vector<BTabEntry> btab;
8     std::vector<ATabEntry> atab;
9     std::vector<int> display; // stack berisi indeks btab per level
10    int level; // lexical level saat ini
11    int dx; // offset penempatan variabel
12 };
  
```

2.4 Inisialisasi: Reserved Slot dan Predefined Identifier

Sesuai spesifikasi (M3 hal. 22), 33 slot pertama **tab** disisihkan untuk *reserved words* dan *predefined identifier*. Implementasi Arion mengisi slot 1–9 dengan tipe, konstanta boolean, serta prosedur built-in seperti dirangkum pada Tabel 2.

Idx	Identifier	Object Class	Keterangan
1	integer	type	predefined simple type
2	real	type	predefined simple type
3	char	type	predefined simple type
4	boolean	type	predefined simple type
5	string	type	predefined simple type
6	true	constant	boolean bernilai 1
7	false	constant	boolean bernilai 0
8	readln	procedure	input dari stdin
9	writeln	procedure	output ke stdout, variadic

Table 2: Predefined identifier yang sudah terdaftar di **tab** sebelum proses analisis dimulai. Identifier user-defined dimulai dari indeks 33.

2.5 Type Compatibility Rules

Tabel 3 merangkum aturan kompatibilitas tipe yang diimplementasikan pada `SymbolTable::compatible()` dan `SymbolTable::assignmentCompatible()`.

Aturan	Deskripsi
Identical type	Dua tipe dengan kode numerik yang sama selalu kompatibel.
Subrange $\equiv$ basis	Subrange dari integer kompatibel dengan integer (baik sebagai sumber maupun tujuan assignment).
Integer $\rightarrow$ Real	integer dapat di-assign ke variabel real (<i>implicit widening</i>).
Subrange $\rightarrow$ Subrange	Dua subrange dengan basis sama selalu kompatibel pada level tipe; pemeriksaan jangkauan dilakukan pada level runtime/code generation.
Anonymous record	Dua record dengan struktur identik tetapi dideklarasikan secara anonim tetap <i>incompatible</i> ; hanya record bernama (<code>type Point = record ...</code>) yang dapat di-assign satu sama lain melalui referensi ref yang sama.
String identity	string kompatibel hanya dengan string .
Boolean operator	and , or , not hanya menerima operand boolean ; hasilnya boolean .
Arithmetic operator	+ , - , * , / hanya menerima operand numerik. Bila salah satu operand real , hasil naik ke real .
div , mod	Hanya menerima operand integer ; hasil integer .
Relational	Operand harus <i>compatible</i> ; hasil selalu boolean .

Table 3: Type compatibility rule yang diimplementasikan pada Arion.

2.6 Semantic Visitor Architecture

Kelas `SemanticAnalyzer` mengimplementasikan *visitor pattern* manual lewat metode `visit(ASTNode*)` yang men-*dispatch* berdasarkan `node->nodeType`. Hirarki pemanggilan utama diperlihatkan pada Figure 7.

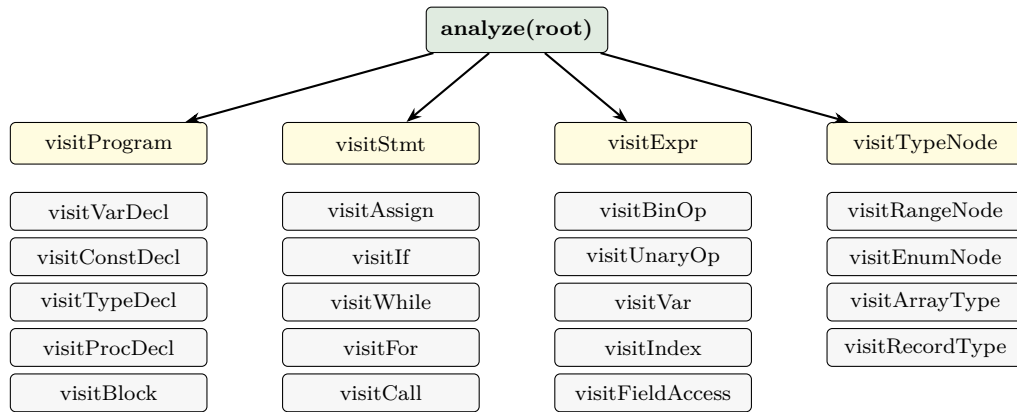


Figure 7: Hirarki pemanggilan visitor function. Empat cabang utama mewakili empat kategori node: deklarasi/program, statement, ekspresi, dan tipe. Setiap kategori disusun vertikal agar tidak terjadi tumpang tindih label.

2.7 Scope Management

Operasi `enterBlock()` dan `exitBlock()` mengelola `stack display` dan `counter level` (lihat Figure 8).

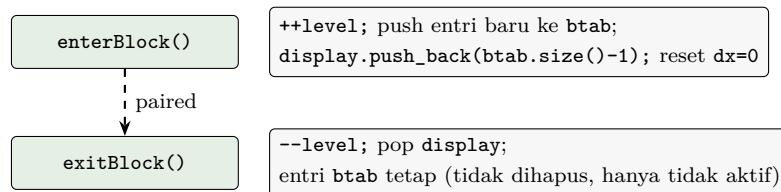


Figure 8: Operasi push/pop scope. Entri pada `btab` tidak pernah dihapus agar referensi (`tab[i].ref`) tetap valid setelah analisis selesai.

2.8 Alur Visit Function untuk Konstruksi Kunci

Diagram pada Figure 9 menggambarkan alur eksekusi `visitAssign` - konstruksi yang melibatkan hampir semua mekanisme semantik: lookup identifier, evaluasi ekspresi, pengecekan tipe, dan anotasi.

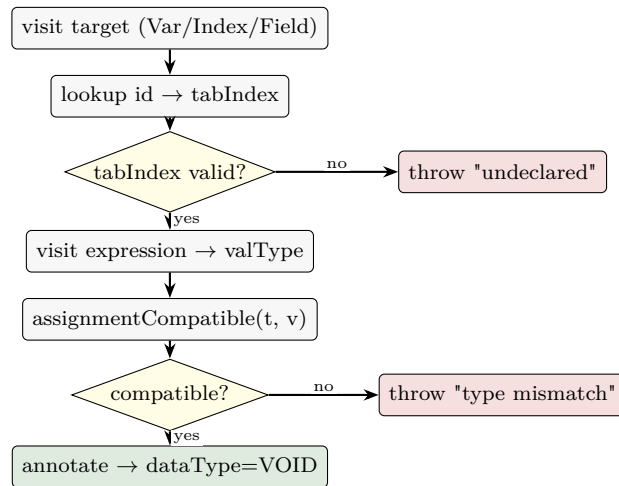


Figure 9: Alur eksekusi `visitAssign`. Pengecekan dilakukan dua tahap: deklarasi target, lalu kompatibilitas tipe sumber dan tujuan.

Implementasi nyata terlihat di kode berikut.

```

1 void SemanticAnalyzer::visitAssign(AssignNode* n) {
2     DataType targetType = T_ERROR;
3     int targetRef = 0;
4     int targetIdx = -1;
5
6     if (n->target->nodeType == AST_VAR) {
7         targetType = visitVar(static_cast<VarNode*>(n->target));
8         targetIdx = n->target->tabIndex;
9         targetRef = (targetIdx >= 0) ? sym.tab[targetIdx].ref : 0;
10    } else if (n->target->nodeType == AST_INDEX) {
11        targetType = visitIndex(static_cast<IndexNode*>(n->target));
12    } else if (n->target->nodeType == AST_FIELDACCESS) {
13        targetType = visitFieldAccess(static_cast<FieldAccessNode*>(n->target));
14    }
15
16    DataType valType = visitExpr(n->value);
17    int valRef = 0;
18    if (n->value->tabIndex >= 0) valRef = sym.tab[n->value->tabIndex].ref;
19
20    if (!sym.assignmentCompatible(targetType, targetRef, valType, valRef)) {
21        throw SemanticError("Semantic error: assignment type mismatch ...");
22    }
23    n->dataType = T_VOID;
24 }

```

2.9 Inferensi Tipe pada Operator Biner

`visitBinOp` menggabungkan empat kategori operator (relasional, boolean, aritmatika, dan integer-only) dengan satu fungsi. Hasil akhirnya diperlihatkan pada Tabel 4.

Operator	Operand	Hasil
<code>==, <>, <, <=, >, >=</code>	Compatible (sama atau salah satu subrange dari yang lain)	<code>boolean</code>
<code>and, or</code>	<code>boolean × boolean</code>	<code>boolean</code>
<code>+, -, *, /</code>	Numerik × Numerik	<code>integer</code> jika kedua <code>integer</code> ; <code>real</code> jika salah satu <code>real</code>
<code>div, mod</code>	<code>integer × integer</code>	<code>integer</code>

Table 4: Aturan inferensi tipe untuk operator biner pada `visitBinOp`.

2.10 Penanganan Prosedur, Fungsi, dan Parameter

Saat memasuki prosedur/fungsi, analyzer memanggil `enterBlock(true)`, mendaftarkan parameter via `enterParameter()` (yang meng-update `btab[i].lpar` sebagai pointer ke parameter terakhir dan menambah `psze`), kemudian memproses deklarasi lokal dan badan blok. Untuk `FuncDeclNode`, sebuah *shadow variable* bernama sama dengan fungsi disisipkan agar assignment `Square := ...` (return value) dapat di-resolve seperti variabel biasa.

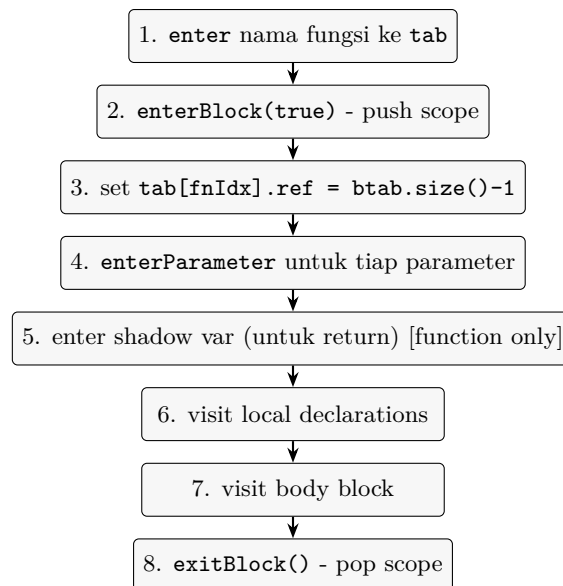


Figure 10: Urutan langkah `visitFuncDecl`. Untuk `visitProcDecl` langkah 5 dilewati. Setelah `exitBlock`, entri `btab` tetap tersimpan agar pemanggilan dari luar dapat me-lookup parameter via `btab[ref].lpar`.

2.11 Error Handling

Setiap pelanggaran semantik dilempar sebagai exception `SemanticError` dengan pesan deskriptif yang menyebut konteks (nama identifier yang salah, tipe yang diharapkan vs yang didapat, dst.). Pada level `main.cpp`, exception ditangkap, di-print ke `stderr`, dan resource (AST + parse tree) dilepaskan secara bersih sebelum program keluar dengan kode 1.

2.12 Format Decorated AST

Decorated AST dicetak dengan format pohon ASCII yang sama dengan parse tree M2, ditambah anotasi inline pada setiap node yang relevan. Format anotasi:

- $\rightarrow \text{tab_index}:N$ - referensi ke entri di tab.
- $\text{type}:T$ - tipe data hasil (untuk ekspresi).
- $\text{lev}:L$ - lexical level (untuk blok yang berpindah scope).

Contoh keluaran lengkap untuk `input-1.txt` (program "Hello"):

Listing 1: Decorated AST + symbol table untuk `test/milestone-3/input-1.txt`.

```

1  --- Decorated AST ---
2  ProgramNode(name: 'Hello')  $\rightarrow$  tab_index:33
3  +-- Block
4  |   \-- VarDecl(a, b)  $\rightarrow$  tab_index:35
5  |       \-- TypeRef(integer)
6  \-- Block, lev:1
7      +-- Assign
8      |   +-- Var(a)  $\rightarrow$  tab_index:34, type:integer
9      |   \-- Num(5), type:integer
10     +-- Assign
11     |   +-- Var(b)  $\rightarrow$  tab_index:35, type:integer
12     |   \-- BinOp(op: '+'), type:integer
13     |       +-- Var(a)  $\rightarrow$  tab_index:34, type:integer
14     |       \-- Num(10), type:integer
15     \-- ProcedureCall(name: 'writeln')  $\rightarrow$  tab_index:9
16         +-- Str('Hasil = '), type:string
17         \-- Var(b)  $\rightarrow$  tab_index:35, type:integer
18  -----
19
20  === tab ===
21  idx      id      obj      type      ref      nrm      lev      adr      link
22  -----
23      0          -          0          0          1          0          0          0
24      1      integer      type          1          0          1          0          0
25      2          real      type          2          0          1          0          1
26      3          char      type          3          0          1          0          2
27      4      boolean      type          4          0          1          0          3
28      5          string      type          5          0          1          0          4
29      6          true      constant      4          0          1          0          1          5
30      7          false      constant      4          0          1          0          0          6
31      8          readln      procedure          0          0          1          0          0          7
32      9          writeln      procedure          0          0          1          0          0          8
33     33          Hello      program          0          0          1          0          0          9
34     34              a      variable          1          0          1          0          0          33
35     35              b      variable          1          0          1          0          0          34
36
37  === btab ===
38  idx  last  lpar  psze  vsze
39  -----
40      0     35      0      0      2
41      1      0      0      0      0
42
43  === atab ===
44  (empty)

```

3 Testing

Pengujian dilakukan terhadap dua kategori berkas masukan di `test/milestone-3/`:

- `input-{n}.txt` - 14 program valid yang harus diproses dengan sukses. Output yang diharapkan adalah **Decorated AST** beserta isi `tab`, `btabs`, dan `atab`.
- `err-{n}.txt` - 11 program yang memuat pelanggaran semantik (mis. *type mismatch*, redeklarasikan, identifikasi tidak dideklarasikan). Output yang diharapkan adalah pesan **Semantic error** dengan deskripsi pelanggaran.

Output lengkap setiap pengujian tersimpan di `test/milestone-3/output/`.

3.1 Ringkasan Kasus Uji Valid

No	Fokus	Deskripsi
1	Golden path	Program minimal dengan <code>var</code> , assignment, dan pemanggilan <code>writeln</code> .
2	Semua tipe simpel	Deklarasi <code>integer</code> , <code>real</code> , <code>char</code> , <code>boolean</code> , <code>string</code> dan operasi pada masing-masing.
3	Scope nested	Prosedur dengan variabel <i>shadowing</i> dan fungsi yang dipanggil dari main.
4	Array + record	<code>const</code> , named record, named array dengan subrange index, akses field dan elemen array.
6	Loop	<code>while</code> dan <code>for-to</code> sederhana.
7	Subrange	Subrange integer (1..10) dan char ('a'..'z').
8	Enumerated	Tipe enumerated (<code>Red</code> , <code>Green</code> , <code>Blue</code>) dan penggunaannya pada <code>assign</code> .
9	String	Beberapa variabel <code>string</code> dan operasi <code>writeln</code> variadic.
10	Scope 3 level	Prosedur dalam prosedur (nested 3 level), uji akses identifikasi dari level terdalam ke level terluar.
11	Ekspresi kompleks	Campuran <code>integer/real</code> , relasional, <code>and</code> , <code>or</code> , dan <code>not</code> .
12	<code>case & repeat</code>	<code>case</code> dengan beberapa arm dan <code>repeat-until</code> dengan ekspresi <code>boolean</code> .
13	Function return	Fungsi rekursif sederhana yang mengembalikan nilai lewat assignment ke nama fungsi.
14	VAR parameter	Prosedur <code>Swap</code> dengan dua <code>var</code> parameter (pass-by-reference).
15	Named record assign	Assignment antar <code>record</code> bernama (record-with-same-named-type kompatibel).

Table 5: Ringkasan 14 kasus uji valid pada Milestone 3.

3.2 Ringkasan Kasus Uji Error

No	Jenis Error	Deskripsi
1	Type mismatch	Assign real ke variabel integer .
2	Redeklarasi	Identifer a dideklarasikan dua kali pada blok yang sama.
3	Type mismatch (bool→int)	Assign boolean ke variabel integer .
4	Array index type	Indeks array bertipe real .
5	Unknown field	Akses field yang tidak ada pada record.
6	If condition bukan bool	Ekspresi if bertipe integer .
7	While condition bukan bool	Ekspresi while bertipe integer .
8	Argument count mismatch	Memanggil prosedur dengan jumlah argumen yang salah.
9	Subrange dari real	type BadRange == 1.0..10.0; (subrange harus ordinal).
10	Anonymous record assign	Assign antar dua record anonim walaupun strukturnya sama.
11	Undeclared identifier	Penggunaan variabel c tanpa deklarasi.

Table 6: Ringkasan 11 kasus uji error pada Milestone 3.

3.3 Hasil Pengujian Valid

Kasus 1 - Golden path

Listing 2: test/milestone-3/input-1.txt

```
program Hello;

var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
  writeln('Hasil = ', b);
end.
```

Listing 3: test/milestone-3/output/output-1.txt

```
1 --- Decorated AST ---
2 ProgramNode(name: 'Hello') → tab_index:33
3 +-- Block
4 |   \-- VarDecl(a, b) → tab_index:35
5 |     \-- TypeRef(integer)
6 \-- Block, lev:1
7   +-- Assign
8   |   +-- Var(a) → tab_index:34, type:integer
9   |   \-- Num(5), type:integer
10  +-- Assign
11  |   +-- Var(b) → tab_index:35, type:integer
12  |   \-- BinOp(op: '+'), type:integer
13  |     +-- Var(a) → tab_index:34, type:integer
14  |     \-- Num(10), type:integer
15  \-- ProcedureCall(name: 'writeln') → tab_index:9
16      +-- Str('Hasil = '), type:string
```

```

17      \-- Var(b) → tab_index:35, type:integer
18      -----
19
20  === tab ===
21      idx          id          obj    type    ref  nrm  lev  adr  link
22      -----
23      0              -          0      0      0    1    0    0    0
24      1      integer      type      1      0    1    0    0    0
25      2          real      type      2      0    1    0    0    1
26      3          char      type      3      0    1    0    0    2
27      4      boolean      type      4      0    1    0    0    3
28      5          string      type      5      0    1    0    0    4
29      6          true      constant  4      0    1    0    1    5
30      7          false      constant  4      0    1    0    0    6
31      8      readln      procedure 0      0    1    0    0    7
32      9      writeln      procedure 0      0    1    0    0    8
33     33      Hello      program    0      0    1    0    0    9
34     34          a      variable    1      0    1    0    0   33
35     35          b      variable    1      0    1    0    0   34
36
37  === btab ===
38      idx  last  lpar  psze  vsze
39      -----
40      0     35    0     0     2
41      1      0    0     0     0
42
43  === atab ===
44  (empty)

```

Kasus 2 - Semua tipe simpel

Listing 4: test/milestone-3/input-2.txt

```

program TypeCheck;

var
  i: integer;
  r: real;
  c: char;
  b: boolean;
  s: string;

begin
  i := 10;
  r := 3.14;
  c := 'X';
  b := true;
  s := 'hello';

  i := i + 5;
  r := r * 2.0;
  b := (i > 0) and (r < 10.0);
  b := not b;

```

```
writeln('Done');  
end.
```

Listing 5: test/milestone-3/output/output-2.txt (45 baris pertama)

```
1  --- Decorated AST ---  
2  ProgramNode(name: 'TypeCheck') → tab_index:33  
3  +--- Block  
4  |   +--- VarDecl(i) → tab_index:34  
5  |   |   \-- TypeRef(integer)  
6  |   +--- VarDecl(r) → tab_index:35  
7  |   |   \-- TypeRef(real)  
8  |   +--- VarDecl(c) → tab_index:36  
9  |   |   \-- TypeRef(char)  
10 |   +--- VarDecl(b) → tab_index:37  
11 |   |   \-- TypeRef(boolean)  
12 |   \-- VarDecl(s) → tab_index:38  
13 |       \-- TypeRef(string)  
14 \-- Block, lev:1  
15   +--- Assign  
16   |   +--- Var(i) → tab_index:34, type:integer  
17   |   \-- Num(10), type:integer  
18   +--- Assign  
19   |   +--- Var(r) → tab_index:35, type:real  
20   |   \-- Num(3.14), type:real  
21   +--- Assign  
22   |   +--- Var(c) → tab_index:36, type:char  
23   |   \-- Char('X'), type:char  
24   +--- Assign  
25   |   +--- Var(b) → tab_index:37, type:boolean  
26   |   \-- Var(true) → tab_index:6, type:boolean  
27   +--- Assign  
28   |   +--- Var(s) → tab_index:38, type:string  
29   |   \-- Str('hello'), type:string  
30   +--- Assign  
31   |   +--- Var(i) → tab_index:34, type:integer  
32   |   \-- BinOp(op: '+'), type:integer  
33   |       +--- Var(i) → tab_index:34, type:integer  
34   |       \-- Num(5), type:integer  
35   +--- Assign  
36   |   +--- Var(r) → tab_index:35, type:real  
37   |   \-- BinOp(op: '*'), type:real  
38   |       +--- Var(r) → tab_index:35, type:real  
39   |       \-- Num(2.0), type:real  
40   +--- Assign  
41   |   +--- Var(b) → tab_index:37, type:boolean  
42   |   \-- BinOp(op: 'and'), type:boolean  
43   |       +--- BinOp(op: '>'), type:boolean  
44   |       |   +--- Var(i) → tab_index:34, type:integer  
45   |       |   \-- Num(0), type:integer
```

Kasus 3 - Scope dengan procedure dan function

Listing 6: test/milestone-3/input-3.txt

```
program ScopeTest;
```

```
var
  x: integer;

procedure Inner;
var
  x: integer;
begin
  x := 100;
  writeln(x);
end;

function Add(a: integer; b: integer): integer;
begin
  Add := a + b;
end;

begin
  x := 1;
  Inner;
  x := Add(x, 5);
  writeln(x);
end.
```

Listing 7: test/milestone-3/output/output-3.txt (45 baris pertama)

```
1 --- Decorated AST ---
2 ProgramNode(name: 'ScopeTest') → tab_index:33
3 +-- Block
4 |   \-- VarDecl(x) → tab_index:34
5 |     \-- TypeRef(integer)
6 +-- ProcDecl(Inner) → tab_index:35
7 |   +-- Block
8 |   |   \-- VarDecl(x) → tab_index:36
9 |   |   |   \-- TypeRef(integer)
10 |   |   \-- Block, lev:1
11 |   |   +-- Assign
12 |   |   |   +-- Var(x) → tab_index:36, type:integer, lev:1
13 |   |   |   |   \-- Num(100), type:integer
14 |   |   |   \-- ProcedureCall(name: 'writeln') → tab_index:9
15 |   |   |   |   \-- Var(x) → tab_index:36, type:integer, lev:1
16 +-- FuncDecl(Add) → tab_index:37
17 |   +-- Param(a) → tab_index:38
18 |   |   \-- TypeRef(integer)
19 |   +-- Param(b) → tab_index:39
20 |   |   \-- TypeRef(integer)
21 |   +-- TypeRef(integer)
22 |   \-- Block, lev:1
23 |   |   \-- Assign
24 |   |   |   +-- Var(Add) → tab_index:40, type:integer, lev:1
25 |   |   |   \-- BinOp(op: '+'), type:integer
26 |   |   |   +-- Var(a) → tab_index:38, type:integer, lev:1
27 |   |   |   \-- Var(b) → tab_index:39, type:integer, lev:1
28 \-- Block, lev:1
```

```

29      +-- Assign
30      |      +-- Var(x) → tab_index:34, type:integer
31      |      \-- Num(1), type:integer
32      +-- ProcedureCall(name: 'Inner') → tab_index:35
33      +-- Assign
34      |      +-- Var(x) → tab_index:34, type:integer
35      |      \-- ProcedureCall(name: 'Add') → tab_index:37, type:integer
36      |      +-- Var(x) → tab_index:34, type:integer
37      |      \-- Num(5), type:integer
38      \-- ProcedureCall(name: 'writeln') → tab_index:9
39      \-- Var(x) → tab_index:34, type:integer
40      -----
41
42  === tab ===
43  idx      id      obj      type      ref      nrm      lev      adr      link
44  -----
45  0              -        0        0        1        0        0        0

```

Kasus 4 - Array dan named record

Listing 8: test/milestone-3/input-4.txt

```

program ArrayRecord;

const
  N == 5;

type
  Point == record
    px: integer;
    py: integer
  end;
  IntArray == array[1..N] of integer;

var
  p: Point;
  arr: IntArray;
  i: integer;

begin
  p.px := 3;
  p.py := 4;
  arr[1] := 10;
  arr[2] := arr[1] + 5;
  writeln('Point: ', p.px, ', ', p.py);
end.

```

Listing 9: test/milestone-3/output/output-4.txt (50 baris pertama)

```

1  --- Decorated AST ---
2  ProgramNode(name: 'ArrayRecord') → tab_index:33

```

```

3  +-- Block
4  |   \-- ConstDecl(N) → tab_index:34
5  |       \-- Num(5), type:integer
6  +-- Block
7  |   +-- TypeDecl(Point) → tab_index:37
8  |   |   \-- RecordType
9  |   \-- TypeDecl(IntArray) → tab_index:38
10 |       \-- ArrayType
11 +-- Block
12 |   +-- VarDecl(p) → tab_index:39
13 |   |   \-- TypeRef(Point)
14 |   +-- VarDecl(arr) → tab_index:40
15 |   |   \-- TypeRef(IntArray)
16 |   \-- VarDecl(i) → tab_index:41
17 |       \-- TypeRef(integer)
18 \-- Block, lev:1
19   +-- Assign
20   |   +-- FieldAccess(px) → tab_index:35, type:integer
21   |   \-- Num(3), type:integer
22   +-- Assign
23   |   +-- FieldAccess(py) → tab_index:36, type:integer
24   |   \-- Num(4), type:integer
25   +-- Assign
26   |   +-- Index, type:integer
27   |   \-- Num(10), type:integer
28   +-- Assign
29   |   +-- Index, type:integer
30   |   \-- BinOp(op: '+'), type:integer
31   |       +-- Index, type:integer
32   |       \-- Num(5), type:integer
33   \-- ProcedureCall(name: 'writeln') → tab_index:9
34       +-- Str('Point: '), type:string
35       +-- FieldAccess(px) → tab_index:35, type:integer
36       +-- Str(', '), type:string
37       \-- FieldAccess(py) → tab_index:36, type:integer
38 -----
39
40 === tab ===
41 idx          id          obj      type    ref  nrm  lev  adr  link
42 -----
43  0              -          0        0      0    1    0    0    0
44  1      integer          type      1      0    1    0    0    0
45  2          real          type      2      0    1    0    0    1
46  3          char          type      3      0    1    0    0    2
47  4      boolean          type      4      0    1    0    0    3
48  5          string        type      5      0    1    0    0    4
49  6          true    constant      4      0    1    0    1    5
50  7          false    constant      4      0    1    0    0    6

```

Kasus 6 - Loop while dan for

Listing 10: test/milestone-3/input-6.txt

```

program LoopTest;

var
  i: integer;

```



```
begin
  i := 0;
  while i < 3 do
    begin
      writeln(i);
      i := i + 1;
    end;

    for i := 1 to 3 do
      begin
        writeln('for ', i);
      end;
    end.
end.
```

Listing 11: test/milestone-3/output/output-6.txt (45 baris pertama)

```
1  --- Decorated AST ---
2  ProgramNode(name: 'LoopTest') → tab_index:33
3  +-- Block
4  |   \-- VarDecl(i) → tab_index:34
5  |       \-- TypeRef(integer)
6  \-- Block, lev:1
7      +-- Assign
8      |   +-- Var(i) → tab_index:34, type:integer
9      |       \-- Num(0), type:integer
10     +-- While
11     \-- For(i, to)
12  -----
13
14  === tab ===
15  idx      id      obj      type      ref      nrm      lev      adr      link
16  -----
17      0          -          0          0          1          0          0          0
18      1      integer      type          1          0          1          0          0
19      2          real      type          2          0          1          0          1
20      3          char      type          3          0          1          0          2
21      4      boolean      type          4          0          1          0          3
22      5          string      type          5          0          1          0          4
23      6          true      constant      4          0          1          0          1
24      7          false      constant      4          0          1          0          0
25      8          readln      procedure      0          0          1          0          0
26      9          writeln      procedure      0          0          1          0          0
27     33      LoopTest      program          0          0          1          0          0
28     34          i          variable      1          0          1          0          0
29
30  === btab ===
31  idx  last  lpar  psze  vsze
32  -----
33      0     34     0     0     1
34      1      0     0     0     0
35
36  === atab ===
37  (empty)
```

Kasus 7 - Subrange types

Listing 12: test/milestone-3/input-7.txt

```

program SubrangeTest;

type
  SmallInt == 1..10;
  Letter == 'a'..'z';

var
  s: SmallInt;
  l: Letter;

begin
  s := 5;
  l := 'm';
  writeln('OK');
end.

```

Listing 13: test/milestone-3/output/output-7.txt (45 baris pertama)

```

1  --- Decorated AST ---
2  ProgramNode(name: 'SubrangeTest') → tab_index:33
3  +--- Block
4  |   +--- TypeDecl(SmallInt) → tab_index:34
5  |   |   \-- Range
6  |   \-- TypeDecl(Letter) → tab_index:35
7  |       \-- Range
8  +--- Block
9  |   +--- VarDecl(s) → tab_index:36
10 |   |   \-- TypeRef(SmallInt)
11 |   \-- VarDecl(l) → tab_index:37
12 |       \-- TypeRef(Letter)
13 \-- Block, lev:1
14   +--- Assign
15   |   +--- Var(s) → tab_index:36, type:subrange
16   |   \-- Num(5), type:integer
17   +--- Assign
18   |   +--- Var(l) → tab_index:37, type:subrange
19   |   \-- Char('m'), type:char
20   \-- ProcedureCall(name: 'writeln') → tab_index:9
21       \-- Str('OK'), type:string
22  -----
23
24  === tab ===
25  idx      id      obj      type      ref      nrm      lev      adr      link
26  -----
27  0          -          0          0          0          1          0          0          0
28  1      integer      type          1          0          1          0          0          0
29  2          real      type          2          0          1          0          0          1
30  3          char      type          3          0          1          0          0          2
31  4      boolean      type          4          0          1          0          0          3
32  5          string      type          5          0          1          0          0          4
33  6          true      constant      4          0          1          0          1          5
34  7          false      constant      4          0          1          0          0          6

```

```

35      8      readln  procedure      0      0      1      0      0      7
36      9      writeln procedure      0      0      1      0      0      8
37     33 SubrangeTest  program      0      0      1      0      0      9
38     34      SmallInt    type      6      0      1      0      0     33
39     35      Letter      type      6      1      1      0      0     34
40     36          s      variable    6      0      1      0      0     35
41     37          l      variable    6      1      1      0      0     36
42
43 === btab ===
44   idx  last  lpar  psze  vsze
45   -----

```

Kasus 8 - Enumerated type

Listing 14: test/milestone-3/input-8.txt

```

program EnumTest;

type
  Color == (Red, Green, Blue);

var
  c: Color;

begin
  c := Red;
  writeln('enum ok');
end.

```

Listing 15: test/milestone-3/output/output-8.txt (40 baris pertama)

```

1  --- Decorated AST ---
2  ProgramNode(name: 'EnumTest') → tab_index:33
3  +-- Block
4  |   \-- TypeDecl(Color) → tab_index:34
5  |       \-- Enum
6  +-- Block
7  |   \-- VarDecl(c) → tab_index:38
8  |       \-- TypeRef(Color)
9  \-- Block, lev:1
10     +-- Assign
11     |   +-- Var(c) → tab_index:38, type:enumerated
12     |   \-- Var(Red) → tab_index:35, type:enumerated
13     \-- ProcedureCall(name: 'writeln') → tab_index:9
14         \-- Str('enum ok'), type:string
15  -----
16
17  === tab ===
18  idx      id      obj  type  ref  nrm  lev  adr  link
19  -----
20      0          -      0      0      1      0      0      0
21      1    integer    type    1      0      1      0      0
22      2      real     type    2      0      1      0      0
23      3      char     type    3      0      1      0      0
24      4    boolean    type    4      0      1      0      0

```

```

25      5      string      type      5      0      1      0      0      4
26      6      true      constant      4      0      1      0      1      5
27      7      false      constant      4      0      1      0      0      6
28      8      readln      procedure      0      0      1      0      0      7
29      9      writeln      procedure      0      0      1      0      0      8
30     33      EnumTest      program      0      0      1      0      0      9
31     34      Color      type      7      0      1      0      0      33
32     35      Red      constant      7      0      1      0      0      34
33     36      Green      constant      7      0      1      0      1      35
34     37      Blue      constant      7      0      1      0      2      36
35     38      c      variable      7      0      1      0      0      37
36
37     === btab ===
38     idx  last  lpar  psze  vsze
39     -----
40      0     38     0     0     4

```

Kasus 9 - String

Listing 16: test/milestone-3/input-9.txt

```

program StringTest;

var
  s1, s2: string;

begin
  s1 := 'hello';
  s2 := 'world';
  writeln(s1, ' ', s2);
end.

```

Listing 17: test/milestone-3/output/output-9.txt (40 baris pertama)

```

1  --- Decorated AST ---
2  ProgramNode(name: 'StringTest') → tab_index:33
3  +--- Block
4  |   \-- VarDecl(s1, s2) → tab_index:35
5  |       \-- TypeRef(string)
6  \-- Block, lev:1
7      +--- Assign
8      |   +--- Var(s1) → tab_index:34, type:string
9      |       \-- Str('hello'), type:string
10     +--- Assign
11     |   +--- Var(s2) → tab_index:35, type:string
12     |       \-- Str('world'), type:string
13     \-- ProcedureCall(name: 'writeln') → tab_index:9
14         +--- Var(s1) → tab_index:34, type:string
15         +--- Char(' '), type:char
16         \-- Var(s2) → tab_index:35, type:string
17  -----
18
19  === tab ===
20  idx      id      obj      type      ref  nrm  lev  adr  link
21  -----

```

```

22      0      -      0      0      1      0      0      0
23      1      integer      type      1      0      1      0      0      0
24      2      real      type      2      0      1      0      0      1
25      3      char      type      3      0      1      0      0      2
26      4      boolean      type      4      0      1      0      0      3
27      5      string      type      5      0      1      0      0      4
28      6      true      constant      4      0      1      0      1      5
29      7      false      constant      4      0      1      0      0      6
30      8      readln      procedure      0      0      1      0      0      7
31      9      writeln      procedure      0      0      1      0      0      8
32      33      StringTest      program      0      0      1      0      0      9
33      34      s1      variable      5      0      1      0      0      33
34      35      s2      variable      5      0      1      0      0      34
35
36      === btab ===
37      idx      last      lpar      psze      vsze
38      -----
39      0      35      0      0      2
40      1      0      0      0      0

```

Kasus 10 - Scope 3 level (nested procedures)

Listing 18: test/milestone-3/input-10.txt

```

program DeepScope;

var
  x: integer;

procedure Outer;
var
  x: integer;
  y: integer;

  procedure Inner;
  var
    z: integer;
  begin
    z := x + 1;
    y := z;
  end;

begin
  x := 10;
  y := 20;
  Inner;
end;

begin
  x := 1;
  Outer;
end.

```

Listing 19: test/milestone-3/output/output-10.txt (50 baris pertama)

```

1  --- Decorated AST ---
2  ProgramNode(name: 'DeepScope') → tab_index:33
3  +-- Block
4  |   \-- VarDecl(x) → tab_index:34
5  |       \-- TypeRef(integer)
6  +-- ProcDecl(Outer) → tab_index:35
7  |   +-- Block
8  |   |   +-- VarDecl(x) → tab_index:36
9  |   |   |   \-- TypeRef(integer)
10 |   |   \-- VarDecl(y) → tab_index:37
11 |   |       \-- TypeRef(integer)
12 |   +-- ProcDecl(Inner) → tab_index:38
13 |   |   +-- Block
14 |   |   |   \-- VarDecl(z) → tab_index:39
15 |   |   |       \-- TypeRef(integer)
16 |   |   \-- Block, lev:2
17 |   |       +-- Assign
18 |   |       |   +-- Var(z) → tab_index:39, type:integer, lev:2
19 |   |       |   \-- BinOp(op: '+'), type:integer
20 |   |       |       +-- Var(x) → tab_index:36, type:integer, lev:1
21 |   |       |       \-- Num(1), type:integer
22 |   |       \-- Assign
23 |   |           +-- Var(y) → tab_index:37, type:integer, lev:1
24 |   |           \-- Var(z) → tab_index:39, type:integer, lev:2
25 |   \-- Block, lev:1
26 |       +-- Assign
27 |       |   +-- Var(x) → tab_index:36, type:integer, lev:1
28 |       |   \-- Num(10), type:integer
29 |       +-- Assign
30 |       |   +-- Var(y) → tab_index:37, type:integer, lev:1
31 |       |   \-- Num(20), type:integer
32 |       \-- ProcedureCall(name: 'Inner') → tab_index:38, lev:1
33 \-- Block, lev:1
34     +-- Assign
35     |   +-- Var(x) → tab_index:34, type:integer
36     |   \-- Num(1), type:integer
37     \-- ProcedureCall(name: 'Outer') → tab_index:35
38 -----
39
40 === tab ===
41   idx      id      obj  type  ref  nrm  lev  adr  link
42   -----
43   0         -       0    0    0    1    0    0    0
44   1    integer     type    1    0    1    0    0    0
45   2      real     type    2    0    1    0    0    1
46   3      char     type    3    0    1    0    0    2
47   4   boolean     type    4    0    1    0    0    3
48   5    string     type    5    0    1    0    0    4
49   6      true  constant    4    0    1    0    1    5
50   7     false  constant    4    0    1    0    0    6

```

Kasus 11 - Ekspresi kompleks

Listing 20: test/milestone-3/input-11.txt

```
program ComplexExpr;

var
  a, b, c: integer;
  r: real;
  flag: boolean;

begin
  a := 1;
  b := 2;
  c := 3;
  r := (a + b) * 2.5;
  flag := (a < b) and (b < c) or (a == c);
  flag := not flag;
  writeln('done');
end.
```

Listing 21: test/milestone-3/output/output-11.txt (45 baris pertama)

```
1  --- Decorated AST ---
2  ProgramNode(name: 'ComplexExpr') → tab_index:33
3  +-- Block
4  |   +-- VarDecl(a, b, c) → tab_index:36
5  |   |   \-- TypeRef(integer)
6  |   +-- VarDecl(r) → tab_index:37
7  |   |   \-- TypeRef(real)
8  |   \-- VarDecl(flag) → tab_index:38
9  |       \-- TypeRef(boolean)
10 \-- Block, lev:1
11   +-- Assign
12   |   +-- Var(a) → tab_index:34, type:integer
13   |   \-- Num(1), type:integer
14   +-- Assign
15   |   +-- Var(b) → tab_index:35, type:integer
16   |   \-- Num(2), type:integer
17   +-- Assign
18   |   +-- Var(c) → tab_index:36, type:integer
19   |   \-- Num(3), type:integer
20   +-- Assign
21   |   +-- Var(r) → tab_index:37, type:real
22   |   \-- BinOp(op: '*'), type:real
23   |       +-- BinOp(op: '+'), type:integer
24   |       |   +-- Var(a) → tab_index:34, type:integer
25   |       |   \-- Var(b) → tab_index:35, type:integer
26   |       \-- Num(2.5), type:real
27   +-- Assign
28   |   +-- Var(flag) → tab_index:38, type:boolean
29   |   \-- BinOp(op: 'or'), type:boolean
30   |       +-- BinOp(op: 'and'), type:boolean
31   |       |   +-- BinOp(op: '<'), type:boolean
32   |       |   |   +-- Var(a) → tab_index:34, type:integer
33   |       |   |   \-- Var(b) → tab_index:35, type:integer
34   |       |   \-- BinOp(op: '<'), type:boolean
35   |       +-- Var(b) → tab_index:35, type:integer
```

```

36 |         |         \-- Var(c) → tab_index:36, type:integer
37 |         \-- BinOp(op: '=='), type:boolean
38 |         +-- Var(a) → tab_index:34, type:integer
39 |         \-- Var(c) → tab_index:36, type:integer
40 +-- Assign
41 | +-- Var(flag) → tab_index:38, type:boolean
42 | \-- UnaryOp(not), type:boolean
43 \-- ProcedureCall(name: 'writeln') → tab_index:9
44 \-- Str('done'), type:string
45 -----

```

Kasus 12 - case dan repeat-until

Listing 22: test/milestone-3/input-12.txt

```

program CaseRepeatTest;

var
  x: integer;

begin
  x := 2;
  case x of
    1: writeln('one');
    2: writeln('two');
    3: writeln('three')
  end;

  repeat
    x := x - 1;
  until x == 0;
end.

```

Listing 23: test/milestone-3/output/output-12.txt (45 baris pertama)

```

1 --- Decorated AST ---
2 ProgramNode(name: 'CaseRepeatTest') → tab_index:33
3 +-- Block
4 | \-- VarDecl(x) → tab_index:34
5 | \-- TypeRef(integer)
6 \-- Block, lev:1
7   +-- Assign
8   | +-- Var(x) → tab_index:34, type:integer
9   | \-- Num(2), type:integer
10   +-- Case
11   \-- Repeat
12 -----
13
14 === tab ===
15   idx      id      obj  type  ref  nrm  lev  adr  link
16 -----
17   0         -      0    0    0    1    0    0    0
18   1  integer  type    1    0    1    0    0    0

```



```

19      2      real      type      2      0      1      0      0      1
20      3      char      type      3      0      1      0      0      2
21      4      boolean    type      4      0      1      0      0      3
22      5      string     type      5      0      1      0      0      4
23      6      true      constant  4      0      1      0      1      5
24      7      false     constant  4      0      1      0      0      6
25      8      readln    procedure 0      0      1      0      0      7
26      9      writeln   procedure 0      0      1      0      0      8
27      33 CaseRepeatTest program  0      0      1      0      0      9
28      34      x        variable  1      0      1      0      0      33
29
30      === btab ===
31      idx  last  lpar  psze  vsze
32      -----
33      0     34    0     0     1
34      1     0     0     0     0
35
36      === atab ===
37      (empty)

```

Kasus 13 - Function dengan return value

Listing 24: test/milestone-3/input-13.txt

```

program FuncReturn;

var
  r: integer;

function Square(n: integer): integer;
begin
  Square := n * n;
end;

begin
  r := Square(5);
  writeln(r);
end.

```

Listing 25: test/milestone-3/output/output-13.txt (45 baris pertama)

```

1  --- Decorated AST ---
2  ProgramNode(name: 'FuncReturn') → tab_index:33
3  +-- Block
4  |   \-- VarDecl(r) → tab_index:34
5  |       \-- TypeRef(integer)
6  +-- FuncDecl(Square) → tab_index:35
7  |   +-- Param(n) → tab_index:36
8  |   |   \-- TypeRef(integer)
9  |   +-- TypeRef(integer)
10 |   \-- Block, lev:1
11 |       \-- Assign
12 |           +-- Var(Square) → tab_index:37, type:integer, lev:1
13 |           \-- BinOp(op: '*'), type:integer

```

```

14 |           +-- Var(n) → tab_index:36, type:integer, lev:1
15 |           \-- Var(n) → tab_index:36, type:integer, lev:1
16 \-- Block, lev:1
17   +-- Assign
18   |   +-- Var(r) → tab_index:34, type:integer
19   |   \-- ProcedureCall(name: 'Square') → tab_index:35, type:integer
20   |       \-- Num(5), type:integer
21   \-- ProcedureCall(name: 'writeln') → tab_index:9
22       \-- Var(r) → tab_index:34, type:integer
23 -----
24
25 === tab ===
26   idx      id      obj  type  ref  nrm  lev  adr  link
27   -----
28   0         -         0    0    0    1    0    0    0
29   1    integer      type    1    0    1    0    0    0
30   2      real      type    2    0    1    0    0    1
31   3      char      type    3    0    1    0    0    2
32   4    boolean      type    4    0    1    0    0    3
33   5      string      type    5    0    1    0    0    4
34   6        true  constant    4    0    1    0    1    5
35   7        false  constant    4    0    1    0    0    6
36   8      readln  procedure    0    0    1    0    0    7
37   9      writeln  procedure    0    0    1    0    0    8
38  33  FuncReturn    program    0    0    1    0    0    9
39  34          r    variable    1    0    1    0    0   33
40  35      Square    function    1    1    1    0    0   34
41  36          n    variable    1    0    1    1    0    0
42  37      Square    variable    1    0    1    1    0   36
43
44 === btab ===
45   idx  last  lpar  psze  vsze

```

Kasus 14 - VAR parameter (pass by reference)

Listing 26: test/milestone-3/input-14.txt

```

program VarParam;

var
  x, y: integer;

procedure Swap(var a: integer; var b: integer);
var
  temp: integer;
begin
  temp := a;
  a := b;
  b := temp;
end;

begin
  x := 1;
  y := 2;

```

```
Swap(x, y);
writeln(x, y);
end.
```

Listing 27: test/milestone-3/output/output-14.txt (45 baris pertama)

```
1  --- Decorated AST ---
2  ProgramNode(name: 'VarParam') → tab_index:33
3  +--- Block
4  |   \-- VarDecl(x, y) → tab_index:35
5  |       \-- TypeRef(integer)
6  +--- ProcDecl(Swap) → tab_index:36
7  |   +--- Param(a, var) → tab_index:37
8  |       |   \-- TypeRef(integer)
9  |   +--- Param(b, var) → tab_index:38
10 |       |   \-- TypeRef(integer)
11 |   +--- Block
12 |       |   \-- VarDecl(temp) → tab_index:39
13 |       |       \-- TypeRef(integer)
14 |   \-- Block, lev:1
15 |       +--- Assign
16 |           |   +--- Var(temp) → tab_index:39, type:integer, lev:1
17 |           |       \-- Var(a) → tab_index:37, type:integer, lev:1
18 |       +--- Assign
19 |           |   +--- Var(a) → tab_index:37, type:integer, lev:1
20 |           |       \-- Var(b) → tab_index:38, type:integer, lev:1
21 |       \-- Assign
22 |           +--- Var(b) → tab_index:38, type:integer, lev:1
23 |           \-- Var(temp) → tab_index:39, type:integer, lev:1
24 \-- Block, lev:1
25     +--- Assign
26     |   +--- Var(x) → tab_index:34, type:integer
27     |       \-- Num(1), type:integer
28     +--- Assign
29     |   +--- Var(y) → tab_index:35, type:integer
30     |       \-- Num(2), type:integer
31     +--- ProcedureCall(name: 'Swap') → tab_index:36
32     |   +--- Var(x) → tab_index:34, type:integer
33     |       \-- Var(y) → tab_index:35, type:integer
34     \-- ProcedureCall(name: 'writeln') → tab_index:9
35         +--- Var(x) → tab_index:34, type:integer
36         \-- Var(y) → tab_index:35, type:integer
37 -----
38
39 === tab ===
40  idx      id      obj  type  ref  nrm  lev  adr  link
41  -----
42    0              -    0    0    1    0    0    0
43    1    integer    type    1    0    1    0    0    0
44    2      real    type    2    0    1    0    0    1
45    3      char    type    3    0    1    0    0    2
```

Kasus 15 - Named record assignment

Listing 28: test/milestone-3/input-15.txt

```
program NamedRecord;
```

```

type
  Point == record x: integer; y: integer; end;

var
  p1, p2: Point;

begin
  p1.x := 1;
  p1.y := 2;
  p2 := p1;
  writeln(p2.x);
end.

```

Listing 29: test/milestone-3/output/output-15.txt (45 baris pertama)

```

1  --- Decorated AST ---
2  ProgramNode(name: 'NamedRecord') → tab_index:33
3  +-- Block
4  |   \-- TypeDecl(Point) → tab_index:36
5  |       \-- RecordType
6  +-- Block
7  |   \-- VarDecl(p1, p2) → tab_index:38
8  |       \-- TypeRef(Point)
9  \-- Block, lev:1
10     +-- Assign
11     |   +-- FieldAccess(x) → tab_index:34, type:integer
12     |   \-- Num(1), type:integer
13     +-- Assign
14     |   +-- FieldAccess(y) → tab_index:35, type:integer
15     |   \-- Num(2), type:integer
16     +-- Assign
17     |   +-- Var(p2) → tab_index:38, type:record
18     |   \-- Var(p1) → tab_index:37, type:record
19     \-- ProcedureCall(name: 'writeln') → tab_index:9
20         \-- FieldAccess(x) → tab_index:34, type:integer
21     -----
22
23  === tab ===
24  idx      id      obj  type  ref  nrm  lev  adr  link
25  -----
26  0         -           0    0    1    0    0    0
27  1      integer      type    1    0    1    0    0    0
28  2         real      type    2    0    1    0    0    1
29  3         char      type    3    0    1    0    0    2
30  4      boolean      type    4    0    1    0    0    3
31  5        string      type    5    0    1    0    0    4
32  6         true  constant    4    0    1    0    1    5
33  7        false  constant    4    0    1    0    0    6
34  8        readln  procedure    0    0    1    0    0    7
35  9        writeln  procedure    0    0    1    0    0    8
36  33  NamedRecord  program    0    0    1    0    0    9
37  34           x      field    1    0    1    0    0    0
38  35           y      field    1    0    1    0    0    34
39  36        Point      type    9    1    1    0    0    33
40  37          p1  variable    9    1    1    0    0    36

```

```
41      38          p2   variable      9      1      1      0      0      37
42
43      === btab ===
44      idx  last  lpar  psze  vsze
45      -----
```

3.4 Hasil Pengujian Error

Pada subbagian ini, setiap testcase memperlihatkan bahwa analyzer **berhasil mendeteksi** pelanggaran semantik dan menghasilkan pesan **Semantic error** yang spesifik.

Error 1 - Assign real ke integer

Listing 30: test/milestone-3/err-1.txt

```
program ErrTypeMismatch;

var
  a: integer;
  b: real;

begin
  a := b;
end.
```

Listing 31: Output - error terdeteksi.

```
Semantic error: assignment type mismatch (cannot assign real to integer)
```

Error 2 - Redeklarasi identifier

Listing 32: test/milestone-3/err-2.txt

```
program ErrRedeclare;

var
  a: integer;
  a: real;

begin
end.
```

Listing 33: Output - error terdeteksi.

```
Semantic error: identifier 'a' is already declared in this scope
```

Error 3 - Assign boolean ke integer

Listing 34: test/milestone-3/err-3.txt

```
program ErrBoolToInt;  
  
var  
  a: integer;  
  b: boolean;  
  
begin  
  a := b;  
end.
```

Listing 35: Output - error terdeteksi.

```
Semantic error: assignment type mismatch (cannot assign boolean to  
integer)
```

Error 4 - Indeks array bertipe real

Listing 36: test/milestone-3/err-4.txt

```
program ErrArrayIndex;  
  
var  
  arr: array[1..10] of integer;  
  idx: real;  
  
begin  
  idx := 1.5;  
  arr[idx] := 5;  
end.
```

Listing 37: Output - error terdeteksi.

```
Semantic error: array index type mismatch
```

Error 5 - Akses field yang tidak ada

Listing 38: test/milestone-3/err-5.txt

```
program ErrUnknownField;  
  
var  
  p: record x: integer; end;
```

```
begin
  p.y := 5;
end.
```

Listing 39: Output - error terdeteksi.

```
Semantic error: unknown field 'y' in record
```

Error 6 - Kondisi if bukan boolean

Listing 40: test/milestone-3/err-6.txt

```
program ErrIfCondition;

var
  a: integer;

begin
  if a then
    a := 1;
end.
```

Listing 41: Output - error terdeteksi.

```
Semantic error: if condition must be boolean
```

Error 7 - Kondisi while bukan boolean

Listing 42: test/milestone-3/err-7.txt

```
program ErrWhileCondition;

var
  a: integer;

begin
  while a do
    begin
      a := 1;
    end;
end.
```

Listing 43: Output - error terdeteksi.

```
Semantic error: while condition must be boolean
```

Error 8 - Jumlah argumen salah

Listing 44: test/milestone-3/err-8.txt

```
program ErrArgCount;  
  
procedure P(a: integer; b: integer);  
begin  
end;  
  
begin  
    P(1);  
end.
```

Listing 45: Output - error terdeteksi.

```
Semantic error: argument count mismatch in call to 'P'
```

Error 9 - Subrange dari real

Listing 46: test/milestone-3/err-9.txt

```
program ErrRealSubrange;  
  
type  
    BadRange == 1.0..10.0;  
  
begin  
end.
```

Listing 47: Output - error terdeteksi.

```
Semantic error: subrange cannot be of type real
```

Error 10 - Assignment antar record anonim

Listing 48: test/milestone-3/err-10.txt

```
program AnonymousRecord;  
  
var  
    point1: record x, y: integer; end;  
    point2: record x, y: integer; end;  
  
begin  
    point1.x := 10;  
    point2 := point1;
```



```
end.
```

Listing 49: Output - error terdeteksi.

```
Semantic error: assignment type mismatch (cannot assign record to record  
)
```

Error 11 - Identifier tidak dideklarasikan

Listing 50: test/milestone-3/err-11.txt

```
program ErrorTest;  
  
var  
  a: integer;  
  b: real;  
  
begin  
  c := 5;           { Error: undeclared identifier c }  
  a := b;           { Error: type mismatch real to integer }  
end.
```

Listing 51: Output - error terdeteksi.

```
Semantic error: undeclared identifier 'c'
```

4 Conclusion

Dari pengerjaan Milestone 3 ini, komponen *Semantic Analyzer* untuk Arion Compiler berhasil diimplementasikan dan diuji sebagai kelanjutan langsung dari parser Milestone 2. Implementasi tetap menggunakan C++17, dengan pendekatan *visitor pattern* manual pada AST yang sebelumnya dikonstruksi dari parse tree melalui `buildAST()`. Secara keseluruhan, program berjalan benar sesuai spesifikasi grammar serta aturan semantik bahasa Arion.

Beberapa hasil utama yang dicapai pada milestone ini adalah sebagai berikut.

- **Symbol table dengan struktur stack-based** yang terdiri atas tiga tabel terpisah (`tab`, `btab`, `atab`) yang saling merujuk, sesuai spesifikasi pada Lampiran B M3.
- **Predefined identifier** (`integer`, `real`, `char`, `boolean`, `string`, `true`, `false`, `readln`, `writeln`) yang sudah terdaftar sebelum analisis dimulai, mengikuti penomoran 33 slot reserved.
- **Decorated AST** yang setiap nodenya dianotasi dengan tipe data, indeks ke `tab`, dan tingkat lexical, siap dikonsumsi oleh tahap *code generation* pada milestone berikutnya.
- **Type compatibility rule** yang lengkap untuk simple type (`integer`, `real`, `char`, `boolean`, `string`), subrange, enumerated, array, dan named/anonymous record.
- **Error handling** dengan exception `SemanticError` yang menyebut tipe error, konteks, dan tipe-tipe yang terlibat, sehingga setiap pelanggaran semantik segera dilaporkan dengan jelas.
- Total **25 kasus uji** (14 valid + 11 error) yang seluruhnya lolos dengan hasil sesuai harapan.

Pencapaian terpenting dari arsitektur ini adalah *pemetaan langsung satu jenis node AST ke satu fungsi visit*. Pendekatan ini menjadikan grammar yang terdokumentasi pada laporan dapat dilacak secara linier ke implementasi C++, tanpa *table-driven* eksternal. Penambahan konstruksi bahasa baru di masa depan cukup berarti menambah satu jenis node serta satu fungsi visit yang sesuai.

4.1 Saran

Berdasarkan hasil pengerjaan Milestone 3 ini, beberapa saran untuk pengembangan lanjutan adalah sebagai berikut.

1. **Error recovery lintas-statement.** Saat ini, satu kesalahan semantik segera menghentikan analisis. Implementasi pemulihan ringan (mis. tandai node `T_ERROR` lalu lanjutkan ke statement berikutnya) akan memungkinkan analyzer melaporkan beberapa error dalam satu eksekusi.
2. **Pengecekan range pada `const` & subrange.** Validasi ranges saat ini terbatas pada literal numerik. Ke depan, evaluasi konstanta yang dideklarasikan via `const` dapat dilakukan *compile-time* sehingga pelanggaran seperti `var i: 1..N where N=5; i := 10` dapat dideteksi lebih awal.

3. **Pelaporan posisi sumber.** Pesan error saat ini menyebut jenis kesalahan dan identifier, tetapi belum konsisten menyertakan nomor baris. Penambahan field **line** pada setiap node dan pewarisannya ke pesan error akan mempermudah *debugging*.
4. **Integrasi dengan code generation.** Decorated AST telah dirancang untuk menjadi masukan tahap berikutnya. Disarankan agar format alamat (**adr**) pada **tab** dipastikan konsisten (mis. *offset stack frame* untuk variabel lokal, *absolute address* untuk variabel global) sejak dini.

References

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.
- [2] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Pearson, 2006.
- [3] P. Geurts, “Semantic Analysis,” Lecture notes, Université de Liège, 2015. <https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>
- [4] R. Spivak, “Let’s Build A Simple Interpreter, Part 7: Abstract Syntax Trees,” <https://ruslanspivak.com/lsbasi-part7/>
- [5] “Abstract Syntax Tree,” CS3203 Course Website, National University of Singapore. <https://nus-cs3203.github.io/course-website/contents/basic-spa-requirements/abstract-syntax-tree.html>
- [6] “Compiler Design - Semantic Analysis,” TutorialsPoint. https://www.tutorialspoint.com/compiler_design/compiler_design_semantic_analysis.htm

Appendix

GitHub Repository

<https://github.com/jsndwrd/DCL-Tubes-IF2224-2026>

Task Distribution

NIM	Nama	Kontribusi
13524026	Made Branenda Jordhy	25%
13524028	Muhammad Nur Majiid	25%
13524034	Jason Edward Salim	25%
13524068	Athilla Zaidan Zidna Fann	25%